

Java Project Structure Example

Directory Structure

```
My2DGame/
├── src/
│   ├── main/
│   │   ├── Main.java      # Entry point with main() method
│   │   ├── GamePanel.java # Custom game panel class
│   │   ├── Player.java    # Custom player class
│   │   └── Enemy.java     # Custom enemy class
│   ├── entities/
│   │   ├── GameObject.java # Base class for game objects
│   │   └── Sprite.java     # Sprite handling class
│   └── utils/
│       ├── InputHandler.java # Input handling utility
│       └── Constants.java    # Game constants
├── bin/                    # VS Code compiled output
│   ├── main/
│   │   ├── Main.class
│   │   ├── GamePanel.class
│   │   ├── Player.class
│   │   └── Enemy.class
│   ├── entities/
│   │   ├── GameObject.class
│   │   └── Sprite.class
│   └── utils/
│       ├── InputHandler.class
│       └── Constants.class
└── out/production/My2DGame/ # IntelliJ compiled output
    ├── main/
    │   ├── Main.class
    │   ├── GamePanel.class
    │   ├── Player.class
    │   └── Enemy.class
    ├── entities/
    │   ├── GameObject.class
    │   └── Sprite.class
    └── utils/
        ├── InputHandler.class
        └── Constants.class
```

File Contents Examples

src/main/Main.java

```
java

package main;

import entities.GameObject;
import utils.Constants;

public class Main {
    public static void main(String[] args) {
        System.out.println("Starting game...");
        GamePanel panel = new GamePanel();
        panel.startGame();
    }
}
```

src/main/GamePanel.java

```
java

package main;

import entities.GameObject;
import entities.Sprite;
import utils.InputHandler;

public class GamePanel {
    private Player player;
    private Enemy enemy;

    public void startGame() {
        player = new Player();
        enemy = new Enemy();
        // Game logic here
    }
}
```

src/main/Player.java

```
java
```

```
package main;

import entities.GameObject;

public class Player extends GameObject {
    private int health;
    private int score;

    public Player() {
        this.health = 100;
        this.score = 0;
    }
}
```

src/entities/GameObject.java

```
java

package entities;

public class GameObject {
    protected int x, y;
    protected int width, height;

    public GameObject() {
        this.x = 0;
        this.y = 0;
    }
}
```

src/utils/Constants.java

```
java

package utils;

public class Constants {
    public static final int SCREEN_WIDTH = 800;
    public static final int SCREEN_HEIGHT = 600;
    public static final int PLAYER_SPEED = 5;
}
```

How to Compile and Run

Manual Compilation (works for both IDEs):

```
bash

# Compile all packages
javac -d bin src/main/*.java src/entities/*.java src/utils/*.java

# Or compile everything at once
javac -d bin src/**/*.java

# Run the main class
java -cp bin main.Main
```

VS Code:

- Automatically compiles to `bin/`
- Click "Run" button above `main()` method
- Or use terminal: `java -cp bin main.Main`

IntelliJ IDEA:

- Automatically compiles to `out/production/My2DGame/`
- Click green "Run" button
- Or use terminal: `java -cp out/production/My2DGame main.Main`

Package Import Rules

```
java

// In src/main/GamePanel.java
package main;           // Current package

import entities.GameObject; // Import from entities package
import utils.Constants;    // Import from utils package
import utils.InputHandler; // Import specific class

// Now you can use GameObject, Constants, InputHandler
```

Key Points

1. **Source files:** Always in `(src/)` with package structure
2. **Compiled files:** Never in `(src/)`, always in output directories
3. **Package declaration:** Must match folder structure
4. **Main method:** Entry point, can be in any class
5. **Imports:** Use package.ClassName format
6. **IDEs:** Handle compilation and classpath automatically